Sprint 06: Refs, Effects, Cleanup, Custom Hooks, සහ Context API

0. Where You Are Before Sprint 06 (Sprint 06 වලට කලින් ඔයා දැනගත යුතු දේවල්)

Sprint 06 පටන් ගන්න කලින්, ඔයාට React වල මේ මූලික සංකල්ප ගැන පැහැදිලි අවබෝධයක් තිබිය යුතුයි:

- React UI හදන්නේ **components** එකතු කරලා.
- **JSX** මඟින් JavaScript ඇතුළේ HTML වගේ syntax ලියන්න පුළුවන්.
- Components වලට දත්ත (data) යවන්නේ **props** හරහායි.
- **children prop** එකෙන් එක component එකක් ඇතුළේ තවත් JSX wrap කරන්න පුළුවන්.
- Lists render කරන්න **map()** පාවිච්චි කරනවා.
- List items නිවැරදිව track කරන්න React වලට **stable keys** අවශ්‍යයි.
- Clicks, typing, සහ form submissions වලට React ප්‍රතිචාර දක්වන්නේ **Events** හරහායි.
- **Controlled inputs** මඟින් form values කෙලින්ම React state එකේ store කරනවා.
- Form එකක් submit වන විට page එක reload වීම වැළැක්වීමට **event.preventDefault()** පාවිච්චි කරනවා.
- වෙනස් වන දත්ත මතක තබා ගැනීමට **useState** පාවිච්චි කරනවා.
- State updates හැමවිටම **immutable** (කෙලින්ම වෙනස් නොකළ හැකි) විය යුතුයි.
- Object state එකක් update කරන විට කලින් තිබූ fields copy කරගත යුතුයි.
- Array state එකක් update කිරීමට spread operator, `filter()`, සහ `map()` වැනි ආරක්ෂිත ක්‍රම පාවිච්චි කළ යුතුයි.
- එකිනෙකට සම්බන්ධ state actions සංවිධානය කරන්න **useReducer** පාවිච්චි කරන්න පුළුවන්.

Sprint 05 වල ප්‍රධාන අවධානය:

$$\text{User action} \rightarrow \text{state changes} \rightarrow \text{React re-renders} \rightarrow \text{UI updates}$$

Sprint 06 වල අලුත් සංකල්පය:

$$\text{React renders UI} \rightarrow \text{effects safely interact with the outside world}$$

සරලවම කිව්වොත්, React component එකක් render වුණාට පස්සේ, ඊට බාහිරින් තියෙන දේවල් (browser DOM, timers, document title, scroll events, reusable hook logic, සහ shared app-level state) එක්ක ආරක්ෂිතව වැඩ කරන්නේ කොහොමද කියලා මේ Sprint එකෙන් ඔයා ඉගෙන ගන්නවා.

1. Sprint 06 Goal (ප්‍රධාන අරමුණු)

1. DOM access කිරීමට සහ re-rendering සිදු නොවන සේ values store කිරීමට **useRef** පාවිච්චි කරන ආකාරය.
2. Side effects සඳහා **useEffect** පාවිච්චි කරන ආකාරය.
3. Timers, intervals, event listeners, සහ subscriptions නිවැරදිව **cleanup** (අයින්) කරන ආකාරය.
4. නැවත නැවත පාවිච්චි කළ හැකි logic සඳහා **custom hooks** හදන ආකාරය.
5. මුළු app එකටම පොදු සරල state එකක් share කරන්න **Context API** පාවිච්චි කරන ආකාරය.

2. Big Mental Model (ප්‍රධාන මානසික ආකෘතිය)

සාමාන්‍යයෙන් React component එකක මූලිකම කාර්යය වෙන්නේ UI එක මොන වගේද කියලා describe කරන එකයි.

උදාහරණයක් ලෙස:

```
return <h1>Hello React</h1>;
```

මේ ක්‍රියාවලියට අපි **rendering** කියනවා.

නමුත් සමහර කාර්යයන් නිකම්ම render කරනවාට වඩා වෙනස්. උදාහරණයක් විදියට:

- Input එකක් focus කිරීම.
- Browser tab title එක වෙනස් කිරීම.
- Timer එකක් start කිරීම / stop කිරීම.
- Scroll event එකකට ඇහුම්කන් දීම (listen).
- Scroll listener එකක් අයින් කිරීම.
- Search input එකක් debounce කිරීම.
- Logged-in user ගේ විස්තර components කිහිපයක් අතර share කිරීම.

මේ වගේ වැඩ කරන්න අපිට විශේෂ Tools අවශ්‍යයි.

Tool	Main job (ප්‍රධාන කාර්යය)
useRef	Re-render කරන්නේ නැතුව DOM elements access කිරීම හෝ mutable values store කිරීම.
useEffect	Rendering ක්‍රියාවලියෙන් පස්සේ side-effect කේතයන් (code) run කිරීම.
Cleanup function	ක්‍රියාත්මක වෙමින් පවතින timers, listeners, හෝ subscriptions නවතා දැමීම.
Custom hook	Hook-based logic නැවත නැවත පාවිච්චි කළ හැකි ලෙස වෙන් කර ගැනීම.
Context API	Prop drilling වලින් තොරව app-level values බෙදාහැරීම (share).

Part One: useRef

3. What Is useRef? (useRef යනු කුමක්ද?)

useRef කියන්නේ React Hook එකක්. එමඟින් අපිට ref object එකක් ලැබෙනවා. Ref object එකක ව්‍යුහය (structure) මේ වගේ සරලයි:

```
{
  current: something
}
```

මෙහි වැදගත්ම කොටස වෙන්නේ **current** කියන property එකයි.

> **සරල අර්ථ දැක්වීම:** Ref එකක් කියන්නේ React renders අතරතුර re-render එකක් සිදු නොකර, යම් අගයක් (value) එකක් ආරක්ෂිතව රඳවා තබා ගන්න පුළුවන් පෙට්ටියක් (box) වගේ දෙයක්.

ප්‍රධානතම භාවිතය: Real DOM element එකක් (උදාහරණයක් විදියට text input එකක්) කෙලින්ම access කිරීමට ref පාවිච්චි කරයි.

භාවිතා වන අවස්ථා:

- Input එකක් focus කිරීමට.
- Uncontrolled input එකක අගය (value) කියවීමට.
- Element එකක ප්‍රමාණය (measurements) මැනීමට.
- UI එක update නොකර යම් කිසි value එකක් මතක තියා ගැනීමට.

4. Basic useRef Syntax (මූලික භාවිතය)

පළමුව React වලින් useRef import කරගන්න:

```
import { useRef } from "react";
```

Ref එකක් නිර්මාණය කරන්න:

```
const inputRef = useRef(null);
```

එය HTML element එකකට සම්බන්ධ (attach) කරන්න:

```
<input ref={inputRef} />
```

`current` property එක හරහා DOM element එක පාවිච්චි කරන්න:

```
inputRef.current.focus();
```

5. useRef Mental Model (මානසික චිත්‍රය)

ඔයා `const inputRef = useRef(null);` කියලා ලියනකොට මෙන්න මේ දේ සිදුවෙනවා:

1. React විසින් පෙට්ටියක් (box) සාදයි.
2. මුලින්ම මේ පෙට්ටිය ඇතුළේ තියෙන්නේ `null` .
3. Input element එක screen එකේ දිස්වුණාට පස්සේ (mount), React විසින් එම real input element එක මේ පෙට්ටිය ඇතුළට දමයි.
4. දැන් ඔයාට `inputRef.current` හරහා එය access කළ හැක.

Timeline එක:

- **Before render:** `inputRef.current = null`
- **After input mounts:** `inputRef.current = actual input element`
- **After component unmounts:** `inputRef.current = null` (නැවතත් හිස් වේ)

6. Example: Focus an Input With useRef

```
import { useRef } from "react";

export default function SearchFocus() {
  const inputRef = useRef(null);

  function focusInput() {
    inputRef.current.focus();
  }

  return (
    <main>
      <h1>Search</h1>
      <input ref={inputRef} placeholder="Search..." />
      <button type="button" onClick={focusInput}>
        Focus input
      </button>
    </main>
  );
}
```

සිදුවන ක්‍රියාවලිය: User බටන් එක click කරයි → `focusInput` function එක run වේ → `inputRef.current` මඟින් input element එක පෙන්වයි → `.focus()` මඟින් input එක focus වේ.

7. Safer Ref Usage (ආරක්ෂිත Ref භාවිතය)

සමහර අවස්ථාවලදී `current` කියන එක `null` වෙන්න පුළුවන්. ඒ නිසා කෙලින්ම `inputRef.current.focus()` ලිවීම අනතුරුදායකයි (error එකක් ආවොත් app එක crash වෙන්න පුළුවන්).

ආරක්ෂිත ක්‍රමය (Safer Version):

```
function focusInput() {
  if (inputRef.current) {
    inputRef.current.focus();
  }
}
```

> **රීතිය:** Ref එක පාවිච්චි කරන්න කලින් සැමවිටම `if (ref.current)` යොදා එය `null` නොවන බව තහවුරු කරගන්න.

8. useRef vs useState

මෙය ඉතාම වැදගත් වෙනසක්!

- **useState** පාවිච්චි කරන්නේ: අගය වෙනස් වන විට screen එකේ UI එක update (re-render) විය යුතු නම් පමණි.
- **useRef** පාවිච්චි කරන්නේ: අගය වෙනස් වන විට UI එක re-render නොවිය යුතු නම් පමණි.

Situation (අවස්ථාව)	පාවිච්චි කළ යුත්තේ කුමක්ද?
Count එකක් screen එකේ පෙන්වීමට සහ එය update කිරීමට	`useState`
Input text එකක් store කර එය live validate කිරීමට	`useState`
Input එකක් focus කිරීමට	`useRef`
DOM element එකක් කෙලින්ම කියවීමට	`useRef`
Timer ID එකක් store කර ගැනීමට	`useRef` (හෝ effect එකක් ඇතුළේ local variable එකක්)
Re-render නොකර අගයක් මතක තබා ගැනීමට	`useRef`

වැරදි Ref භාවිතයක් (Bad Use of Ref):

```
const countRef = useRef(0);
```

ජීවිතේට screen එකේ පෙන්වන count එකක් මේ විදියට හදන්න එපා, මොකද countRef වෙනස් වුණාට screen එක update වෙන්නේ නැහැ.

නිවැරදි ක්‍රමය:

```
const [count, setCount] = useState(0);
```

9. Refs and Uncontrolled Inputs (Refs සහ Uncontrolled Inputs)

Uncontrolled input එකක් කියන්නේ input එකේ අගය React state එකක store කරන්නේ නැතුව, browser DOM එකෙන්ම කෙලින්ම store වෙන්න ඉඩ හරින එකටයි.

```
import { useRef } from "react";

export default function ReferralForm() {
  const refCodeInput = useRef(null);

  function handleSubmit(event) {
    event.preventDefault();
    const refCode = refCodeInput.current.value;
    console.log("Referral code:", refCode);
  }

  return (
    <form onSubmit={handleSubmit}>
      <label htmlFor="refCode">Referral code</label>
      <input id="refCode" ref={refCodeInput} />
      <button type="submit">Submit</button>
    </form>
  );
}
```

Uncontrolled inputs පාවිච්චි කරන්නේ:

- Form එක submit කරන වෙලාවට විතරක් අගය අවශ්‍ය නම්.
- Live validation (ටයිප් කරන ගමන් වැරදි සෙවීම) අවශ්‍ය නැත්නම්.
- Live preview (ටයිප් කරන දේ එවලේම screen එකේ පෙන්වීම) අවශ්‍ය නැත්නම්.

නමුත්, සාමාන්‍ය *React* වැඩ වලදී *Controlled inputs (useState පාවිච්චි කරන ඒවා)* තමයි හොඳම සහ *default* විකල්පය වෙන්නේ.

Part Two: useEffect

10. What Is useEffect? (useEffect යනු කුමක්ද?)

useEffect කියන්නේ React render වෙන පද්ධතියෙන් බාහිරව සිදුවන වැඩ (side effects) හැසිරවීමට ගන්නා React Hook එකක්.

> **සරල අර්ථ දැක්වීම:** React විසින් UI එක සාර්ථකව render කර අවසන් වූ පසුව යම් code එකක් run කිරීමට `useEffect` උපකාරී වේ.

Side Effects සඳහා උදාහරණ:

- `document.title` එක වෙනස් කිරීම.
- Timer එකක් හෝ Interval එකක් පටන් ගැනීම.
- Event listener එකක් ඇතුළත් කිරීම (add) හෝ ඉවත් කිරීම (remove).
- API එකකින් data fetch කිරීම.
- LocalStorage වල දත්ත save කිරීම.
- Server එකක් එක්ක connect වීම (WebSockets).

11. Why Effects Exist (Effects පවතින්නේ ඇයි?)

React rendering ක්‍රියාවලිය සාමාන්‍යයෙන් **pure** විය යුතුයි. Pure rendering කියන්නේ: එකම props සහ state දුන්නොත්, component එක හැමතිස්සෙම එකම UI එකක් return කළ යුතුයි. rendering ඇතුළේ බාහිර ලෝකය වෙනස් කරන වැඩ කරන්න බැහැ.

- **වැරදි ක්‍රමය:** Render වෙන ගමන් component එක ඇතුළේ කෙලින්ම API calls, timers වගේ බාහිර දේවල් ක්‍රියාත්මක කිරීම.
- **නිවැරදි ක්‍රමය:** මුලින්ම UI එක render වෙන්න ඉඩ හරින්න. ඊට පස්සේ render එක ඉවර වුණාම `useEffect` ඇතුළේ බාහිර වැඩ (side-effects) සිදු කරන්න.

12. Basic useEffect Syntax (මූලික ව්‍යුහය)

```
import { useEffect } from "react";

useEffect(() => {
  // side effect කේතය මෙතැන ලියන්න
}, [dependencies]);
```

කොටස	අර්ථය
useEffect	Effects හැසිරවීමට ගන්නා React Hook එක
`() => {}`	Effect එක ක්‍රියාත්මක වන විට run විය යුතු function එක
Dependency array	මේ effect එක නැවත run විය යුත්තේ කවදාද යන්න පාලනය කරන array එක

13. Example: Update document.title

```
import { useEffect, useState } from "react";

export default function TitleUpdater() {
  const [name, setName] = useState("");

  useEffect(() => {
    document.title = name ? `Hello, ${name}` : "React App";
  }, [name]);

  return (
    <main>
      <h1>Title Updater</h1>
      <label htmlFor="name">Name</label>
      <input
        id="name"
        value={name}
        onChange={(event) => setName(event.target.value)}
      />
    </main>
  );
}
```

සිදුවන ක්‍රියාවලිය: User ටයිප් කරයි → `name` state එක වෙනස් වේ → React නැවත render වේ → render එක ඉවර වූ පසු `useEffect` ක්‍රියාත්මක වේ → `document.title` එක අලුත් නම සමඟ update වේ.

Part Three: Dependency Arrays

14. What Is a Dependency Array? (Dependency Array එකක් යනු කුමක්ද?)

Dependency Array එකක් මඟින් React වලට පවසන්නේ මෙම effect එක නැවත ක්‍රියාත්මක කළ යුත්තේ කවදාද යන්නයි. එය `useEffect` එකේ අගට යොදනු ලබයි.

ප්‍රධාන අවස්ථා 3ක් ඇත:

1. Dependency array එකක් **නොමැති** අවස්ථාව (No dependency array).
2. **හිස්** dependency array එකක් ඇති අවස්ථාව (Empty dependency array `[]`).
3. **අගයන් සහිත** dependency array එකක් ඇති අවස්ථාව (Dependency array with values).

15. Case 1: No Dependency Array (Array එකක් නැති විට)

```
useEffect(() => {
  console.log("Effect ran");
});
```

අර්ථය: සෑම render එකකටම පසුව මෙම effect එක run වේ. ප්‍රවේශම් වන්න: Component එකේ state එකක් වෙනස් වන සෑම වාරයකදීම මෙය අනවශ්‍ය ලෙස නැවත නැවත run වේ. සාමාන්‍යයෙන් මෙය භාවිත කරන්නේ ඉතාම කලාතුරකිනි.

16. Case 2: Empty Dependency Array (හිස් Array එකක් ඇති විට `[]`)

```
useEffect(() => {
  console.log("Effect ran once");
}, []);
```

අර්ථය: Component එක screen එකේ පළමු වතාවට දිස්වන විට (mount) එක් වතාවක් පමණක් ක්‍රියාත්මක වේ. පොදු භාවිතයන්:

- මුල්ම වතාවේ data load (fetch) කිරීම.
- Event listeners register කිරීම.
- Component එක load වන විට යමක් ආරම්භ කිරීම.

17. Case 3: Dependency Array With Values (අගයන් සහිත විට)

```
useEffect(() => {
  console.log("Query changed:", query);
}, [query]);
```

අර්ථය:

- මුල්ම render එකෙන් පසුව ක්‍රියාත්මක වේ.
- ඉන්පසු, `query` වල අගය වෙනස් වන සෑම අවස්ථාවකදීම නැවත ක්‍රියාත්මක වේ.

> **රන් රීතිය (Memory Rule):** ඔයාගේ effect එක ඇතුළේ component එකේ තියෙන යම් reactive value එකක් (props, state, හෝ ඇතුළේ හදපු variables/functions) පාවිච්චි කරනවා නම්, එය අනිවාර්යයෙන්ම dependency array එකට ඇතුළත් කළ යුතුය.

18. Dependency Array Summary (කෙටි සාරාංශය)

Dependency Style	ක්‍රියාත්මක වන්නේ කවදාද?
No array	හැම render එකකටම පස්සේ
`[]`	පළමු render එකෙන් පස්සේ එක් වතාවක් පමණි
`[value]`	පළමු render එකෙන් පස්සේ සහ `value` වෙනස් වන හැම වාරයකදීම
`[a, b]`	පළමු render එකෙන් පස්සේ සහ `a` හෝ `b` වෙනස් වන හැම වාරයකදීම

නිතර සිදුවන වැරැද්දක් (Common Dependency Mistake):

```
// වැරදි ක්‍රමය
useEffect(() => {
  console.log(query);
}, []); // query පාවිච්චි කරලා තියෙද්දි dependency එක හිස් කරලා!
```

නිවැරදි ක්‍රමය:

```
useEffect(() => {
  console.log(query);
}, [query]);
```

Part Four: Cleanup Functions

20. What Is Cleanup? (Cleanup යනු කුමක්ද?)

Cleanup function එකක් කියන්නේ `useEffect` එක ඇතුළෙන් අපි return කරන තවත් function එකකටයි. එමඟින් effect එක මඟින් ආරම්භ කළ දේවල් නවතා දැමීම හෝ ඉවත් කිරීම සිදු කරයි.

මූලික හැඩය (Basic Pattern):

```
useEffect(() => {
  // යමක් ආරම්භ කිරීම (start something)

  return () => {
    // ආරම්භ කළ දේ නැවත නවතා දැමීම (stop that thing)
  };
}, []);
```

> **සරල අර්ථ දැක්වීම:** Cleanup function එක මඟින් පරණ side effects දිගටම ක්‍රියාත්මක වෙමින් පැවතීම නවතා දමයි.

21. Why Cleanup Is Important (Cleanup වැදගත් වන්නේ ඇයි?)

සමහර effects මඟින් දිගින් දිගටම පසුබිමෙන් ක්‍රියාත්මක වන වැඩ ආරම්භ කරයි. උදාහරණ:

- `setTimeout()`
- `setInterval()`
- Scroll / Resize listeners
- Subscriptions

අපි ඒවා නිසි පරිදි cleanup කළේ නැත්නම් මේ වගේ bugs ඇති වෙන්න පුළුවන්:

- එකම timer එක කිහිප සැරයක් duplicate වීම.
- Memory leaks (මතකය අනවශ්‍ය ලෙස වැය වීම).
- Console logs නැවත නැවත අනවශ්‍ය ලෙස print වීම.
- Component එක screen එකෙන් අයින් වුණාට පස්සෙත් පසුබිමෙන් code ක්‍රියාත්මක වීම.

22. Cleanup With setTimeout

```
useEffect(() => {
  const id = setTimeout(() => {
    console.log("Timer finished");
  }, 1000);

  return () => {
    clearTimeout(id); // component එක අයින් වුණොත් හෝ රීලෝඩ් වුණොත් ටයිමරය
නවත්වයි
  };
}, []);
```

23. Cleanup With setInterval

```
useEffect(() => {
  const id = setInterval(() => {
    console.log("Interval running");
  }, 1000);

  return () => {
    clearInterval(id); // interval එක නවත්වයි
  };
}, []);
```

24. Cleanup With Event Listeners

```
import { useEffect, useState } from "react";

export default function ScrollMessage() {
  const [scrolled, setScrolled] = useState(false);

  useEffect(() => {
    function handleScroll() {
      setScrolled(true);
    }

    window.addEventListener("scroll", handleScroll);

    return () => {
      window.removeEventListener("scroll", handleScroll); // listener එක
අයින් කරයි
    };
  }, []);

  return (
    <main style={{ minHeight: "150vh" }}>
      <h1>Scroll Listener</h1>
      {scrolled && <p>You scrolled.</p>}
    </main>
  );
}
```

> **රීතිය:** Effect එකක් ඇතුළේ `addEventListener` එකක් දැම්මොත්, අනිවාර්යයෙන්ම cleanup එක ඇතුළේ `removeEventListener` එකක් දාන්න.

25. Effect Lifecycle Mental Model (Effect එකක ජීවන චක්‍රය)

Dependencies වෙනස් වන විට සාමාන්‍යයෙන් සිදුවන්නේ මෙයයි:

1. **Cleanup old effect** (කලින් තිබූ effect එක අයින් කරයි).
2. **Run new effect** (අලුත් effect එක run කරයි).

Component එක screen එකෙන් සම්පූර්ණයෙන්ම ගැලවී යන විට (unmount):

- **Cleanup final effect** (අවසාන effect එකත් අයින් කර දමයි).

Part Five: Debouncing and Delayed Preview

26. What Is Debouncing? (Debouncing යනු කුමක්ද?)

Debouncing කියන්නේ පරිශීලකයා (user) යම් ක්‍රියාවක් කරලා (උදා: typing) එය සම්පූර්ණයෙන්ම නවත්වන තෙක් සුළු වෙලාවක් බලා සිට පසුව logic එක run කිරීමයි.

උදාහරණයක් ලෙස: Search box එකක user අකුරක් අකුරක් පාසා ටයිප් කරන සෑම වාරයකම API එකට requests යවන්නේ නැතිව, user ටයිප් කර ඉවර වී තත්පර බාගයක් (500ms) බලා සිට පසුව search එක සිදු කිරීම.

වාසි:

- අනවශ්‍ය API calls ප්‍රමාණය අඩු වීම.
- App එකේ වේගය සහ smoothness එක වැඩි වීම.

27. Example: Delayed Preview

```
import { useEffect, useState } from "react";

export default function DelayedPreview() {
  const [text, setText] = useState("");
  const [preview, setPreview] = useState("");

  useEffect(() => {
    const id = setTimeout(() => {
      setPreview(text);
    }, 700);

    return () => {
      clearTimeout(id); // user නැවත ටයිප් කරන විට කලින් තිබූ timeout එක නවතී
    };
  }, [text]);

  return (
    <main>
      <h1>Delayed Preview</h1>
      <label htmlFor="message">Message</label>
      <input
        id="message"
        value={text}
        onChange={(event) => setText(event.target.value)}
      />
      <p>Instant text: {text}</p>
      <p>Delayed preview: {preview}</p>
    </main>
  );
}
```

Part Six: Custom Hooks

28. What Is a Custom Hook? (Custom Hook එකක් යනු කුමක්ද?)

Custom hook එකක් කියන්නේ React වල තියෙන hooks (useState, useEffect වැනි) පාවිච්චි කරලා අපි විසින්ම සාදාගන්නා වෙනම reusable (නැවත නැවත භාවිත කළ හැකි) function එකකටයි.

> **සරල අර්ථ දැක්වීම:** Components කිහිපයකටම එකම hook-based logic එකක් අවශ්‍ය වන විට, එය එකම තැනක ලියා ගැනීමට custom hooks උදව් වේ.

සෑමවිටම *custom hooks* වල නම `use` අකුරෙන් ආරම්භ විය යුතුය (*උදා:* `useDebounce`, `useFetch`).

29. Why Custom Hooks Must Start With "use"

React වල hooks වලට අදාළ විශේෂ නීති (Rules of Hooks) ඇත. නම `use` වලින් පටන් ගැනීම මඟින් React tools වලට සහ linter එකට හඳුනාගැනීමට පුළුවන් මෙය සාමාන්‍ය function එකක් නොව React Hook එකක් බව.

- **නිවැරදි නම:** `useDebounce`
- **වැරදි නම:** `debounceValue`

30. What Logic Should Become a Custom Hook? (Custom Hook එකක් බවට පත් කළ යුත්තේ කුමක්ද?)

- එකම state/effect logic එක components කිහිපයක නැවත ලියන්න වෙනවා නම්.
- Component එකක් ඇතුළේ logic එක ඕනෑවට වඩා සංකීර්ණ වී පිරී තිබෙනවා නම්.
- එම logic එකෙන් කෙලින්ම JSX render කරන්නේ නැතිනම්.

31. Build a useDebounce Custom Hook (useDebounce custom hook එක හදමු)

File path: `src/hooks/useDebounce.js`

```
import { useEffect, useState } from "react";

export function useDebounce(value, delay) {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    const id = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);

    return () => {
      clearTimeout(id);
    };
  }, [value, delay]);

  return debouncedValue;
}
```

33. Use useDebounce in a SearchBox

```
import { useEffect, useRef, useState } from "react";
import { useDebounce } from "./hooks/useDebounce";

export default function SearchBox() {
  const inputRef = useRef(null);
  const [query, setQuery] = useState("");
  const debouncedQuery = useDebounce(query, 500);

  useEffect(() => {
    document.title = debouncedQuery ? `Search: ${debouncedQuery}` :
"Search";
  }, [debouncedQuery]);

  function focusInput() {
    if (inputRef.current) {
      inputRef.current.focus();
    }
  }

  return (
    <main>
      <h1>Search Box</h1>
      <label htmlFor="search">Search</label>
      <input
        id="search"
        ref={inputRef}
        value={query}
        onChange={(event) => setQuery(event.target.value)}
        placeholder="Type to search..."
      />
      <button type="button" onClick={focusInput}>
        Focus input
      </button>
      <p>Instant query: {query}</p>
      <p>Debounced query: {debouncedQuery}</p>
    </main>
  );
}
```

Part Seven: Context API

34. What Problem Does Context Solve? (Context මඟින් විසඳන ප්‍රශ්නය කුමක්ද?)

සාමාන්‍යයෙන් React වල data pass කරන්නේ parent component එකේ සිට child component එකට props හරහායි. නමුත් සමහර අවස්ථාවලදී deep child component එකකට (ගොඩක් යටින් තියෙන component එකකට) data එකක් යවන්න, අතරමැද තියෙන හැම component එකකටම prop එකක් විදියට pass කරන්න සිදුවේ.

උදාහරණයක් ලෙස:

$$\text{App} \rightarrow \text{Layout} \rightarrow \text{Header} \rightarrow \text{Navbar} \rightarrow \text{UserMenu}$$

මෙතනදී `UserMenu` එකට විතරක් `user` කියන data එක අවශ්‍ය වුණත්, අතරමැද තියෙන Layout, Header, Navbar හැම එකක් හරහාම එය pass කරන්න වෙනවා. මේ ප්‍රශ්නයට අපි කියනවා **Prop Drilling** කියලා.

> **සරල අර්ථ දැක්වීම:** Prop Drilling කියන්නේ component එකකට අදාළ දත්ත අවශ්‍ය නොවුණත්, යටින්ම තියෙන වෙනත් component එකකට එම දත්ත යැවීම සඳහා අතරමැදියෙකු ලෙස ක්‍රියා කරමින් props pass කිරීමයි.

35. What Is Context API? (Context API යනු කුමක්ද?)

Context API කියන්නේ React වල තියෙන built-in feature එකක්. එමඟින් props pass කරන්නේ නැතුව කෙලින්ම අවශ්‍ය ඕනෑම component එකකට දත්ත ලබා දිය හැකියි.

පොදු භාවිතයන්:

- දැනට ලොග් වෙලා ඉන්න පරිශීලකයාගේ විස්තර (Current user).
- Theme එක (Light / Dark mode).
- භාෂාව (Language - Sinhala / English).
- Shopping cart summary එක.

වැදගත්: හැම පොඩි state එකක්ම Context එකට දාන්න එපා. Local state හැමවිටම component එක ඇතුළේම තබා ගන්න.

36. Context API Has Three Main Pieces (ප්‍රධාන කොටස් 3ක්)

1. `createContext()` : Shared data බහාලන පෙට්ටිය සාදයි.
2. `Provider` : මෙම දත්තයන් යටින් තියෙන සියලුම components වලට ලබා දේ (make available).
3. `useContext()` : ලබා දුන් දත්ත කියවීමට (read) child components වලදී පාවිච්චි කරයි.

37. Piece 1: createContext()

```
import { createContext } from "react";
const AuthContext = createContext(null); // null යනු default අගයයි
```

38. Piece 2: Provider

```
<AuthContext.Provider value={value}>
  {children}
</AuthContext.Provider>
```

39. Piece 3: useContext()

```
import { useContext } from "react";
const auth = useContext(AuthContext);
```

40. Full Auth Context Example (සම්පූර්ණ උදාහරණයක්)

File path: `src/context/AuthContext.jsx`

```
import { createContext, useContext, useState } from "react";

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);

  function login(name) {
    setUser({ name });
  }

  function logout() {
    setUser(null);
  }

  const value = {
    user,
    login,
    logout,
  };

  return (
    <AuthContext.Provider value={value}>
      {children}
    </AuthContext.Provider>
  );
}

// Custom hook එකක් හදලා context එක ලේසියෙන් පාවිච්චි කරන්න
export function useAuth() {
  return useContext(AuthContext);
}
```

42. Navbar Using Auth Context

File path: `src/components/Navbar.jsx`

```
import { useAuth } from "../context/AuthContext";

export function Navbar() {
  const { user, login, logout } = useAuth();

  return (
    <nav>
      {user ? (
        <>
          <p>Logged in as {user.name}</p>
          <button type="button" onClick={logout}>
            Logout
          </button>
        </>
      ) : (
        <button type="button" onClick={() => login("Sangeeth")}>
          Login
        </button>
      )}
    </nav>
  );
}
```

43. Use the Provider in App.jsx

```
import { useState } from "react";
import { AuthProvider } from "./context/AuthContext";
import { Navbar } from "./components/Navbar";

export default function App() {
  const [localNote, setLocalNote] = useState("");

  return (
    <AuthProvider>
      <main>
        <Navbar />
        <h1>Sprint 06 Practice</h1>
        <label htmlFor="note">Local note</label>
        <input
          id="note"
          value={localNote}
          onChange={(event) => setLocalNote(event.target.value)}
        />
        <p>This local note does not need Context: {localNote}</p>
      </main>
    </AuthProvider>
  );
}
```

44. Context Tree Mental Model (මානසික ව්‍යුහය)

$$\text{App} \rightarrow \text{AuthProvider} \rightarrow \text{Navbar (reads from Context)} \rightarrow \text{Dashboard (reads fr}$$

- `localNote` එක App component එකට පමණක් සීමා වන නිසා එය Context එකට දැමීමට අවශ්‍ය නැත (Keep local state local).
- `user` එක app එකේ විවිධ තැන් වලට අවශ්‍ය වන නිසා එය Context එකේ තබයි.

Part Eight: Debug Task

46. Broken Code (වැරදි සහිත කේතය)

```
useEffect(() => {
  const id = setInterval(() => console.log(query), 1000);
}, []);

function focusInput() {
  inputRef.current.focus();
}
```

ප්‍රධාන වැරදි 3:

1. **Missing Cleanup:** `setInterval` එකක් පටන් ගත්තත් එය නවත්වන්න (clearInterval) cleanup එකක් නැත.
2. **Missing Dependency:** Effect එක ඇතුළේ `query` පාවිච්චි කර ඇතත්, dependency array එක හිස්ව `[]` ඇත.
3. **Unsafe Ref Access:** `inputRef.current` එක `null` විය හැකි අවස්ථාවකදී කෙලින්ම `.focus()` කිරීම අනතුරුදායකය.

50. Fixed Debug Code (නිවැරදි කළ කේතය)

```
useEffect(() => {
  const id = setInterval(() => {
    console.log(query);
  }, 1000);

  return () => {
    clearInterval(id); // Cleanup fixed!
  };
}, [query]); // Dependency fixed!

function focusInput() {
  if (inputRef.current) {
    inputRef.current.focus(); // Safe access fixed!
  }
}
```

Part Nine: Common Sprint 06 Mistakes (නිතර සිදුවන වැරදි)

Mistake 1: UI එකට Ref පාවිච්චි කිරීම (Using Ref instead of State for UI Data)

- **Wrong:** `const countRef = useRef(0);` (Screen එකේ පෙන්වන දේවල් වලට refs ගන්න එපා).
- **Better:** `const [count, setCount] = useState(0);`

Mistake 2: ref.current එක check නොකිරීම (Not checking ref.current)

- **Wrong:** `inputRef.current.focus();`
- **Better:** `if (inputRef.current) { inputRef.current.focus(); }`

Mistake 3: Dependency Array එක අමතක වීම (Forgetting the Dependency Array)

- **Wrong:** `useEffect(() => { document.title = query; });` (සෑම render එකකදීම අනවශ්‍ය ලෙස run වේ).
- **Better:** `useEffect(() => { document.title = query; }, [query]);`

Mistake 4: State එකක් මත යැපෙන විට හිස් array `[]` දැමීම

- **Wrong:** `useEffect(() => { console.log(query); }, []);`
- **Better:** `useEffect(() => { console.log(query); }, [query]);`

Mistake 5: Timers වලට Cleanup නොදැමීම (Missing Cleanup for Timers)

- **Wrong:** `useEffect(() => { setTimeout(() => { console.log("Done"); }, 1000); }, []);`
- **Better:**

  ```
  useEffect(() => {
    const id = setTimeout(() => { console.log("Done"); }, 1000);
    return () => clearTimeout(id);
  }, []);
  ```

Mistake 6: Event Listeners වලට Cleanup නොදැමීම

- **Wrong:** `useEffect(() => { window.addEventListener("scroll", handleScroll); }, []);`
- **Better:**

  ```
  useEffect(() => {
    window.addEventListener("scroll", handleScroll);
    return () => window.removeEventListener("scroll", handleScroll);
  }, []);
  ```

Mistake 7: Custom Hook එකක නම "use" වලින් පටන් නොගැනීම

- **Wrong:** `function debounce(value, delay) { ... }`
- **Better:** `function useDebounce(value, delay) { ... }`

Mistake 8: Context ඕනෑවට වඩා භාවිත කිරීම (Overusing Context)

- **Wrong:** හැම පොඩි state එකක්ම Context එක ඇතුළට දැමීම.
- **Better:** Local state එක component එක ඇතුළේම තියාගෙන, app-level දේවල් විතරක් Context එකට දාන්න.

Part Ten: Practice Projects

Practice Project 1: Ref Focus App

Goal: බටන් එක click කළ විට input එක focus වන සරල app එකක්.

```
import { useRef } from "react";

export default function RefFocusApp() {
  const inputRef = useRef(null);

  function handleFocus() {
    if (inputRef.current) {
      inputRef.current.focus();
    }
  }

  return (
    <main>
      <h1>Ref Focus App</h1>
      <label htmlFor="search">Search</label>
      <input id="search" ref={inputRef} />
      <button type="button" onClick={handleFocus}>
        Focus input
      </button>
    </main>
  );
}
```

Practice Project 2: Document Title App

Goal: ටයිප් කරන දේ අනුව browser title එක වෙනස් කිරීම.

```
import { useEffect, useState } from "react";

export default function DocumentTitleApp() {
  const [title, setTitle] = useState("");

  useEffect(() => {
    document.title = title || "React App";
  }, [title]);

  return (
    <main>
      <h1>Document Title App</h1>
      <label htmlFor="title">Page title</label>
      <input
        id="title"
        value={title}
        onChange={(event) => setTitle(event.target.value)}
      />
    </main>
  );
}
```

Practice Project 3: Delayed Preview App

Goal: ටයිප් කර තත්පර 0.7ක් (700ms) බලා සිටි පසු preview එක පෙන්වීම.

```
import { useEffect, useState } from "react";

export default function DelayedPreviewApp() {
  const [text, setText] = useState("");
  const [preview, setPreview] = useState("");

  useEffect(() => {
    const id = setTimeout(() => {
      setPreview(text);
    }, 700);

    return () => {
      clearTimeout(id);
    };
  }, [text]);

  return (
    <main>
      <h1>Delayed Preview</h1>
      <label htmlFor="text">Text</label>
      <input
        id="text"
        value={text}
        onChange={(event) => setText(event.target.value)}
      />
      <p>Instant: {text}</p>
      <p>Delayed: {preview}</p>
    </main>
  );
}
```

Part Eleven: Revision Questions and Answers (ප්‍රශ්න හා පිළිතුරු)

Q1: useRef එකක් සහ useState එකක් අතර වෙනස කුමක්ද?

- **useRef** පාවිච්චි කරන්නේ අගය වෙනස් වන විට component එක re-render කිරීමට අවශ්‍ය නැති අවස්ථා වලදීය (උදා: DOM access, timers).
- **useState** පාවිච්චි කරන්නේ අගය වෙනස් වන විට screen එකේ UI එක අලුත් (re-render) විය යුතු අවස්ථා වලදීය.

Q2: useRef(null) කියන එකෙන් අදහස් කරන්නේ කුමක්ද?

එයින් අදහස් කරන්නේ ආරම්භයේදී (initial render එකේදී) කිසිදු element එකක් සම්බන්ධ වී නැති (null අගයක් සහිත) ref object එකක් නිර්මාණය කරන බවයි. Element එක render වූ පසු React විසින් real DOM element එක `ref.current` වලට ආදේශ කරයි.

Q3: useEffect පාවිච්චි කරන්නේ ඇයි?

React rendering ක්‍රියාවලියෙන් පසුව ක්‍රියාත්මක විය යුතු බාහිර වැඩ (side effects - API fetching, timers, document title update) සිදු කිරීමටයි.

Q4: හිස් dependency array `[]` එකකින් අදහස් කරන්නේ කුමක්ද?

Component එක පළමු වතාවට screen එකේ පෙන්වන විට (mount) එම effect එක එක් වතාවක් පමණක් ක්‍රියාත්මක වන බවයි.

Q5: Dependency array එකක් නැති විට (`useEffect` එකක අගට array එකක් නොලියූ විට) සිදුවන්නේ කුමක්ද?

Component එක render වන හෝ re-render වන සෑම වාරයකදීම එම effect එක නැවත නැවත ක්‍රියාත්මක වේ.

Q6: Dependency array එකක `[query]` ලෙස ලියා තිබීමේ තේරුම කුමක්ද?

පළමු render එකෙන් පසුව සහ ඉන්පසු `query` state එක වෙනස් වන සෑම අවස්ථාවකදීම මෙම effect එක නැවත ක්‍රියාත්මක වන බවයි.

Q7: Cleanup function එකක් අනිවාර්යයෙන්ම අවශ්‍ය වන්නේ ඇයි?

Component එකක් අයින් වන විට (unmount) හෝ ඊළඟ effect එක run වීමට පෙර, කලින් තිබූ timers, intervals, event listeners වැනි දේ නවතා දැමීමටයි. නැතහොත් memory leaks සහ duplicate bugs ඇති විය හැක.

Q8: Context API එකක් මඟින් විසඳන ප්‍රධාන ප්‍රශ්නය කුමක්ද?

Prop Drilling ප්‍රශ්නයයි. එනම් අතරමැද ඇති අනවශ්‍ය components හරහා props pass නොකර, ඕනෑම component එකකට කෙලින්ම පොදු දත්ත ලබා දීමයි.

Q9: Context API එකක ප්‍රධාන කොටස් 3 මොනවාද?

1. `createContext()`
2. `Provider`
3. `useContext()`

Q10: Context API එක useState වෙනුවට සම්පූර්ණයෙන්ම පාවිච්චි කළ හැකිද?

නැත. Context යනු state store කරන තැනක් නොවේ, එය state එකක් share කරන මාර්ගයකි. තවද, සියලුම states Context එකේ දැමීමෙන් අනවශ්‍ය re-renders බහුලව සිදුවේ. Local state හැමවිටම local විය යුතුය.

Part Thirteen: Definition of Done (ස්වයං ඇගයීම් ලැයිස්තුව)

ඔයා මේ දේවල් තනියම කරන්න පුළුවන් නම් ඔයා Sprint 06 සාර්ථකව අවසන් කර ඇත:

- [ ] useRef ක්‍රියා කරන ආකාරය පැහැදිලි කිරීමට හැකි වීම.
- [ ] useRef මඟින් input එකක් focus කිරීමට හැකි වීම.
- [ ] useEffect මඟින් document.title එක update කිරීමට හැකි වීම.
- [ ] Dependency array රටා (patterns) 3 පැහැදිලි කිරීමට හැකි වීම.
- [ ] setTimeout සහ setInterval සඳහා cleanup function ලිවීමට හැකි වීම.
- [ ] Event listener එකක් add කර එය නිවැරදිව remove කිරීමට හැකි වීම.
- [ ] useDebounce custom hook එකක් සාදා එය search box එකක භාවිත කිරීමට හැකි වීම.
- [ ] සරල AuthProvider එකක් සාදා login/logout state එක handle කිරීමට හැකි වීම.